

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:

Akmallullail Sya'ban NIM. 2310817310010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Akmallullail Sya'ban
NIM : 2310817310010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 1993070320190301011

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	8
B. Output Program	16
C. Pembahasan	18
D. Tautan Git	22
SOAL 2.....	23
A. Pembahasan	23

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi.....	6
Gambar 2. Tampilan Pilihan Persentase Tip	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan	7
Gambar 4. Tampilan Awal Aplikasi Tip Calculator XML.....	16
Gambar 5. Tampilan Persentase Tip Calculator XML	16
Gambar 6. Tampilan Aplikasi Setelah Dijalankan	17
Gambar 7. Tampilan Awal Aplikasi Tip Compose	17
Gambar 8. Tampilan Pilihan Persentasi Tip Compose.....	18
Gambar 9. Tampilan Aplikasi Setelah Dijalankan	18

DAFTAR TABEL

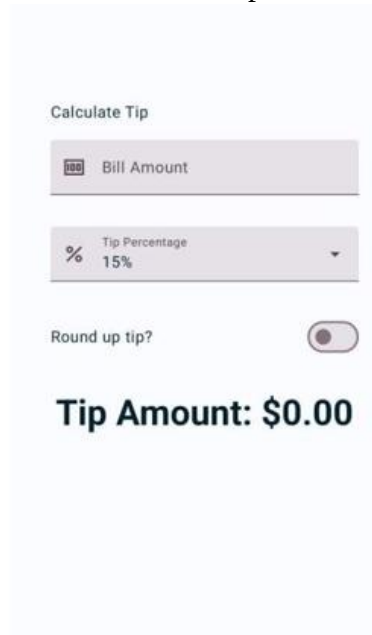
Tabel 1. Source Code Jawaban Soal 1 XML.....	9
Tabel 2. Source Code Jawaban Soal 1 XML.....	11
Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose.....	15

SOAL 1

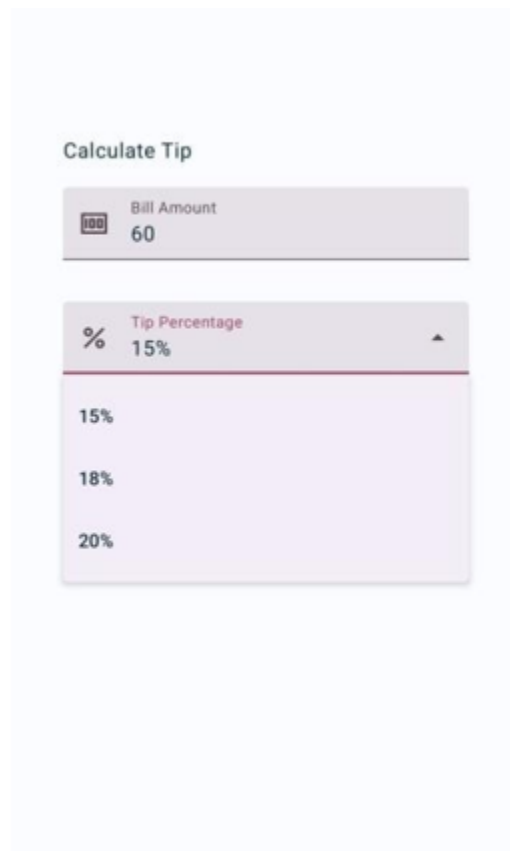
Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

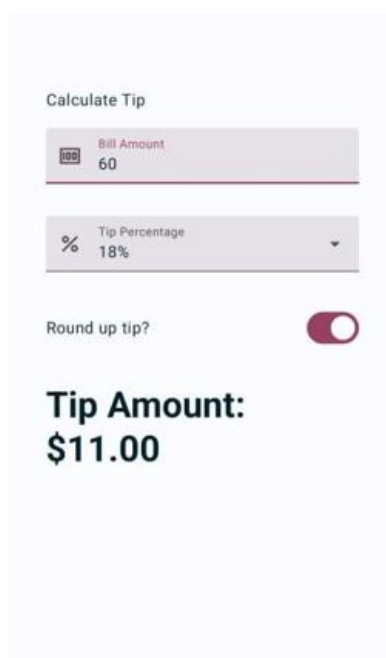
- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

XML:

MainActivity.kt

```

1 package com.example.tipcalculator
2
3 import android.os.Bundle
4 import android.text.Editable
5 import android.text.TextWatcher
6 import android.widget.ArrayAdapter
7 import androidx.appcompat.app.AppCompatActivity
8 import
9     com.example.tipcalculator.databinding.ActivityMainBinding
10 import java.text.NumberFormat
11 import kotlin.math.ceil
12
13 class MainActivity : AppCompatActivity() {
14     private lateinit var binding: ActivityMainBinding
15     private val tipOptions = listOf("15%", "18%", "20%")
16
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         binding = ActivityMainBinding.inflate(layoutInflater)
20         setContentView(binding.root)
21         val adapter = ArrayAdapter(this,
22             android.R.layout.simple_dropdown_item_1line, tipOptions)
23
24         binding.autoCompleteTip.setAdapter(adapter)
25         binding.autoCompleteTip.setText("15%", false)
26         binding.autoCompleteTip.setOnClickListener {
27             binding.autoCompleteTip.showDropDown()
28         }
29         binding.autoCompleteTip.setOnItemClickListener { _,
30             _, _ ->
31                 updateTip()
32             }
33         binding.etBill.addTextChangedListener(object :
34             TextWatcher {
35                 override fun afterTextChanged(s: Editable?) =
36                     updateTip()
37                 override fun beforeTextChanged(s: CharSequence?,
38                     start: Int, count: Int, after: Int) {}
39                 override fun onTextChanged(s: CharSequence?,
40                     start: Int, before: Int, count: Int) {}
41             })
42         binding.switchRound.setOnCheckedChangeListener { _, _
43             -> updateTip() } }
44     private fun updateTip() {
45         val amount =
46             binding.etBill.text.toString().toDoubleOrNull() ?: 0.0

```



```

39         val tipText = binding.autoCompleteTip.text.toString()
40         val tipPercent =
tipText.removeSuffix("%").toDoubleOrNull() ?: 15.0
41         val roundUp = binding.switchRound.isChecked
42         val tip = calculateTip(amount, tipPercent, roundUp)
43         binding.tvTotalTip.text =
getString(R.string.tip_amount, tip)
44     }
45     private fun calculateTip(amount: Double, tipPercent:
Double, roundUp: Boolean): String {
46         var tip = tipPercent / 100 * amount
47         if (roundUp) {
48             tip = ceil(tip)
49         }
50         return NumberFormat.getCurrencyInstance().format(tip)
51     }
52 }

```

Tabel 1. Source Code Jawaban Soal 1 XML

activity_main.xml

```

1  <?xml                      version="1.0"                      encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:padding="24dp"
8      tools:context=".MainActivity">
9
10     <TextView
11         android:id="@+id/textViewLabel"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:layout_marginTop="120dp"
15         android:text="@string/calculate_tip"
16         android:textSize="18sp"
17         app:layout_constraintBottom_toTopOf="@id/etBill"
18         app:layout_constraintEnd_toEndOf="parent"
19         app:layout_constraintHorizontal_bias="0.067"
20         app:layout_constraintStart_toStartOf="parent"
21         app:layout_constraintTop_toTopOf="parent"
22     />
23
24     <EditText
25         android:id="@+id/etBill"
26         android:layout_width="0dp"
27         android:layout_height="48dp"
28         android:layout_marginTop="2dp"
29         android:hint="@string/bill_amount"
30         android:background="#ffe9f4"

```

```

31         android:autofillHints=""
32         android:paddingStart="8dp"
33         android:drawableStart="@drawable/money"
34         android:drawablePadding="8dp"
35         android:inputType="numberDecimal"
36         app:layout_constraintTop_toBottomOf="@+id/textViewLabel"
37         app:layout_constraintStart_toStartOf="parent"
38         app:layout_constraintEnd_toEndOf="parent"
39         app:layout_constraintHorizontal_bias="0.5"
40         app:layout_constraintWidth_percent="0.9"
41     />
41
42     <com.google.android.material.textfield.TextInputLayout
43         android:id="@+id/tipInputLayout"
44     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
45         android:layout_width="0dp"
46         android:background="#ffe9f4"
47         app:boxBackgroundMode="outline"
48         android:layout_height="wrap_content"
49         android:layout_marginTop="16dp"
50         app:layout_constraintTop_toBottomOf="@id/etBill"
51         app:layout_constraintStart_toStartOf="parent"
52         app:layout_constraintEnd_toEndOf="parent"
53         app:layout_constraintWidth_percent="0.9">
54
55         <AutoCompleteTextView
56             android:id="@+id/autoCompleteTip"
57             android:layout_width="match_parent"
58             android:layout_height="48dp"
59             android:drawableStart="@drawable/persen"
60             android:drawablePadding="8dp"
61             android:paddingStart="8dp"
62         android:drawableEnd="@drawable/baseline_arrow_drop_down_24"
63             android:hint="@string/tip_percentage"
64             android:inputType="none"
65         />
66
67     </com.google.android.material.textfield.TextInputLayout>
68
69     <androidx.appcompat.widget.SwitchCompat
70         android:id="@+id/switchRound"
71         android:layout_width="0dp"
72         android:layout_height="48dp"
73         android:layout_marginTop="30dp"
74         android:text="@string/round_up_tip"
75         app:layout_constraintTop_toBottomOf="@id/tipInputLayout"
76         app:layout_constraintStart_toStartOf="parent"
77         app:layout_constraintEnd_toEndOf="parent"
78         app:layout_constraintWidth_percent="0.9"
79     />

```



```

32 import androidx.compose.material3.Text
33 import androidx.compose.material3.TextField
34 import androidx.compose.runtime.Composable
35 import androidx.compose.runtime.getValue
36 import androidx.compose.runtime.mutableStateOf
37 import androidx.compose.runtime.remember
38 import androidx.compose.runtime.setValue
39 import androidx.compose.ui.Alignment
40 import androidx.compose.ui.Modifier
41 import androidx.compose.ui.res.painterResource
42 import androidx.compose.ui.res.stringResource
43 import androidx.compose.ui.text.input.ImeAction
44 import androidx.compose.ui.text.input.KeyboardType
45 import androidx.compose.ui.tooling.preview.Preview
46 import androidx.compose.ui.unit.dp
47 import
com.example.tipcalculatorcompose.ui.theme.TipTimeTheme
import java.text.NumberFormat

48
49 class MainActivity : ComponentActivity() {
50     override fun onCreate(savedInstanceState: Bundle?) {
51         enableEdgeToEdge()
52         super.onCreate(savedInstanceState)
53         setContent {
54             TipTimeTheme{
55                 Surface(
56                     modifier = Modifier.fillMaxSize(),
57                 ) {
58                     TipTimeLayout()
59                 }
60             }
61         }
62     }
63 }
64
65 @OptIn(ExperimentalMaterial3Api::class)
66 @Composable
67 fun TipTimeLayout() {
68     var amountInput by remember { mutableStateOf("") }
69     var roundUp by remember { mutableStateOf(false) }
70
71     val options = listOf("15%", "18%", "20%")
72     var selectedOptionText by remember {
mutableStateOf(options[0]) }
73     var expanded by remember { mutableStateOf(false) }
74
75     val amount = amountInput.toDoubleOrNull() ?: 0.0
76     val tipPercent =
selectedOptionText.removeSuffix("%").toDoubleOrNull() ?:
15.0
77     val tip = calculateTip(amount, tipPercent, roundUp)
78

```

```

79     Column(
80         modifier = Modifier
81             .statusBarsPadding()
82             .padding(horizontal = 40.dp)
83             .verticalScroll(rememberScrollState())
84             .safeDrawingPadding(),
85         horizontalAlignment = Alignment.CenterHorizontally,
86         verticalArrangement = Arrangement.Center
87     ) {
88         Text(
89             text = stringResource(R.string.calculate_tip),
90             modifier = Modifier
91                 .padding(bottom = 16.dp, top = 40.dp)
92                 .align(alignment = Alignment.Start)
93         )
94         EditNumberField(
95             label = R.string.bill_amount,
96             leadingIcon = R.drawable.money,
97             keyboardOptions = KeyboardOptions.Default.copy(
98                 keyboardType = KeyboardType.Number,
99                 imeAction = ImeAction.Next
100         ),
101         value = amountInput,
102         onValueChanged = { amountInput = it },
103         modifier = Modifier.padding(bottom =
104             32.dp).fillMaxWidth(),
105         )
106         ExposedDropDownMenuBox(
107             expanded = expanded,
108             onExpandedChange = {expanded = !expanded},
109             modifier = Modifier
110                 .padding(bottom = 32.dp)
111                 .fillMaxWidth()
112         ) {
113             TextField(
114                 readOnly = true,
115                 value = selectedOptionText,
116                 onValueChange = {},
117                 label = { Text("Tip Percentage") },
118                 trailingIcon = {
119                     ExposedDropDownMenuDefaults.TrailingIcon(expanded =
120                         expanded)
121                 },
122                 modifier = Modifier
123                     .menuAnchor()
124                     .fillMaxWidth(),
125                 colors =
126                     ExposedDropDownMenuDefaults.textFieldColors()
127             )
128             ExposedDropDownMenu(
129                 expanded = expanded,

```

```

127         onDismissRequest = { expanded = false}
128     ){
129         options.forEach{ selectionOption ->
130             DropdownMenuItem(
131                 text = { Text(selectionOption)},
132                 onClick = {
133                     selectionOption
134                     expanded = false
135                 }
136             )
137         }
138     }
139 }
140 }
141
142 RoundTheTipRow(
143     roundUp = roundUp,
144     onRoundUpChanged = { roundUp = it },
145     modifier = Modifier.padding(bottom = 32.dp)
146 )
147 Text(
148     text = stringResource(R.string.tip_amount, tip),
149     style = MaterialTheme.typography.displaySmall,
150     modifier = Modifier.align(Alignment.Start)
151 )
152 Spacer(modifier = Modifier.height(150.dp))
153 }
154 }
155
156 @Composable
157 fun EditNumberField(
158     @StringRes label: Int,
159     @DrawableRes leadingIcon: Int,
160     keyboardOptions: KeyboardOptions,
161     value: String,
162     onValueChanged: (String) -> Unit,
163     modifier: Modifier = Modifier
164 ) {
165     TextField(
166         value = value,
167         singleLine = true,
168         leadingIcon = { Icon(painter = painterResource(id =
169             leadingIcon), null) },
170         modifier = modifier,
171         onValueChange = onValueChanged,
172         label = { Text(stringResource(label)) },
173         keyboardOptions = keyboardOptions
174     )
175 }

```

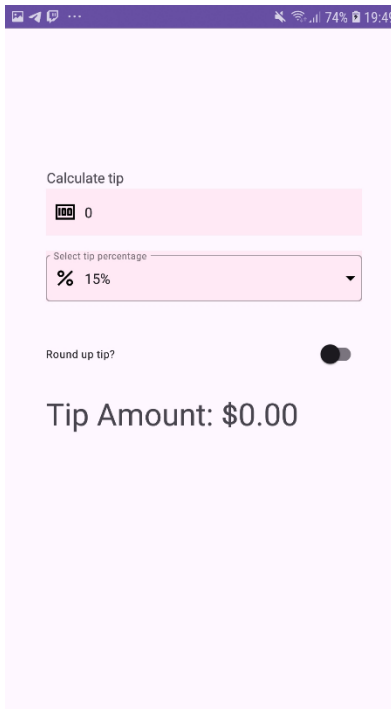
```

176 @Composable
177 fun RoundTheTipRow(
178     roundUp: Boolean,
179     onRoundUpChanged: (Boolean) -> Unit,
180     modifier: Modifier = Modifier
181 ) {
182     Row(
183         modifier = modifier.fillMaxWidth(),
184         verticalAlignment = Alignment.CenterVertically
185     ) {
186         Text(text = stringResource(R.string.round_up_tip))
187         Switch(
188             modifier = Modifier
189                 .fillMaxWidth()
190                 .wrapContentWidth(Alignment.End),
191             checked = roundUp,
192             onCheckedChange = onRoundUpChanged
193         )
194     }
195 }
196
197 private fun calculateTip(amount: Double, tipPercent: Double
198 = 15.0, roundUp: Boolean): String {
199     var tip = tipPercent / 100 * amount
200     if (roundUp) {
201         tip = kotlin.math.ceil(tip)
202     }
203     return NumberFormat.getCurrencyInstance().format(tip)
204 }
205
206 @Preview(showBackground = true)
207 @Composable
208 fun TipTimeLayoutPreview() {
209     TipTimeTheme{
210         TipTimeLayout()
211     }
212 }

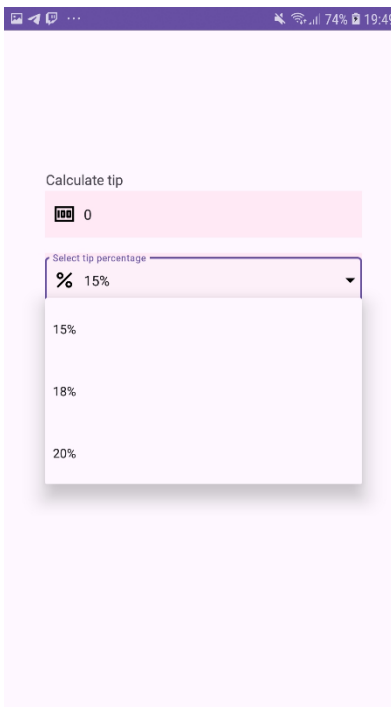
```

Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose

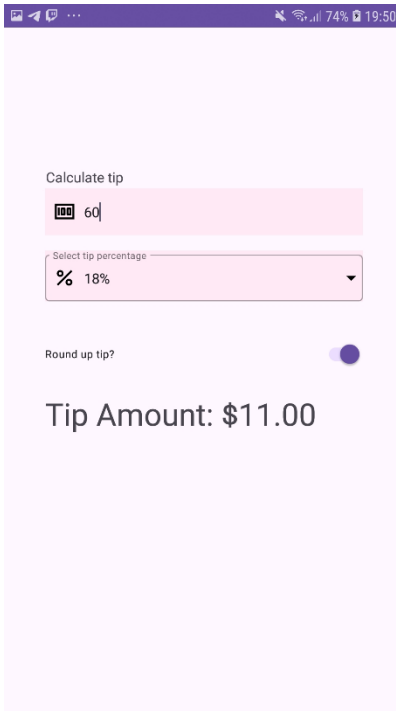
B. Output Program



Gambar 4. Tampilan Awal Aplikasi Tip Calculator XML



Gambar 5. Tampilan Persentase Tip Calculator XML



Calculate tip

100 60

Select tip percentage

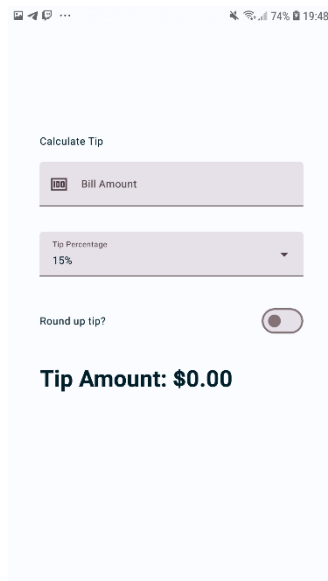
% 18%

Round up tip? ☒

Tip Amount: \$11.00

This screenshot shows the app's interface after a calculation. The background is a light pink color. At the top, there's a status bar with icons for signal, Wi-Fi, and battery (74%) at 19:50. Below the status bar, the text "Calculate tip" is displayed. There are two input fields: the first contains "100" and the second contains "60". Below these is a dropdown menu labeled "Select tip percentage" with the selected option being "% 18%". A toggle switch for "Round up tip?" is turned on. The final result, "Tip Amount: \$11.00", is displayed in a large, bold font.

Gambar 6. Tampilan Aplikasi Setelah Dijalankan



Calculate Tip

150 Bill Amount

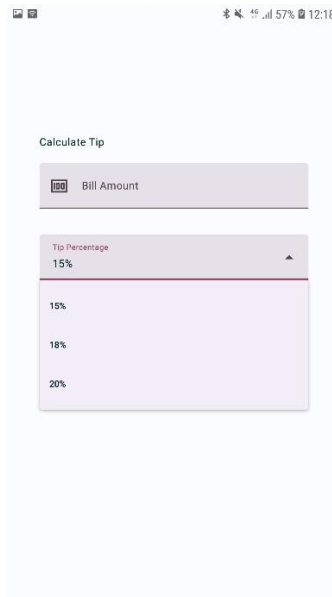
Tip Percentage
15%

Round up tip? ☐

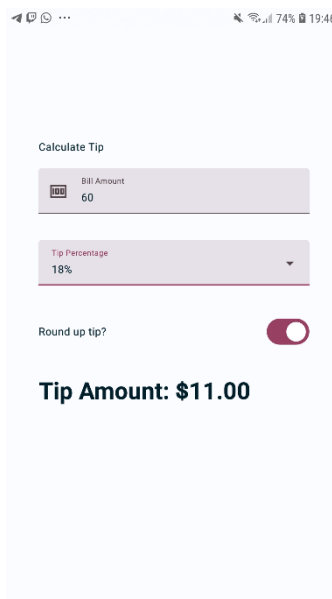
Tip Amount: \$0.00

This screenshot shows the app's initial state. The background is a light blue color. At the top, there's a status bar with icons for signal, Wi-Fi, and battery (74%) at 19:48. Below the status bar, the text "Calculate Tip" is displayed. There are two input fields: the first contains "150" and is labeled "Bill Amount", and the second is empty and labeled "Tip Percentage". Below these is a dropdown menu with the selected option being "15%". A toggle switch for "Round up tip?" is turned off. The final result, "Tip Amount: \$0.00", is displayed in a large, bold font.

Gambar 7. Tampilan Awal Aplikasi Tip Compose



Gambar 8. Tampilan Pilihan Persentasi Tip Compose



Gambar 9. Tampilan Aplikasi Setelah Dijalankan

C. Pembahasan

XML:

MainActivity.kt:

Pada baris [1], package com.example.tipcalculator ini menentukan package dari file ini

Pada baris [3 – 10], ini berisikan import import yang digunakan pada logika ini, import perubahan teks, menghubungkan list data ke AutoCompleteTextView, base class untuk fitur fitur, view binding, memformat angka jadi bentuk mata uang, dan membulatkan angka keatas

Pada baris [12], `class MainActivity : AppCompatActivity() {` ini mewarisi dari `AppCompatActivity`

Pada baris [14], `private lateinit var binding: ActivityMainBinding` ini adalah properti view binding untuk akses view.

Pada baris [15], `private val tipOptions = listOf("15%", "18%", "20%")` ini list persentase tip untuk dropdown.

Pada baris [17 – 18], ini adalah fungsi yang pertama kali dipanggil saat activity dibuat.

Pada baris [19], `binding = ActivityMainBinding.inflate(layoutInflater)` ini merupakan inisialisasi view binding

Pada baris [20], `setContentView(binding.root)` menampilkan layout utama dari binding

Pada baris [21], baris ini membuat adapter untuk menghubungkan list `tipOptions` ke dropdown `AutoCompleteTextView`

Pada baris [23], baris ini mengatur adapter ke `AutoCompleteTextView`

Pada baris [24], baris ini menetapkan nilai awal saat aplikasi dibuka yaitu 15%

Pada baris [25 – 27], ini baris yang membuat binding dapat menampilkan dropdown saat di klik

Pada baris [28 – 30], ini baris yang membuat user saat memilih item dari dropdown langsung update nilai tip

Pada baris [31 – 35], menambahkan listener untuk melihat perubahan teks pada `EditText`. Dimana ketika teks berubah langsung hitung ulang tip.

Pada baris [36], membuat switch pembulatan yang jika di on/off langsung hitung ulang nilai tip

Pada baris [37 – 44], pada blok kode ini membuat fungsi menghitung dan menampilkan tip. Mulai dari nilai bill yang dikonversi ke double dan default nya kosong. Teks persentase dari dropdown. Menghapus % lalu ubah ke double jika default 15.0 gagal. Mengambil status switch apakah perlu dibulatkan atau tidak. Hitung nilai tip berdasarkan input dan kondisi pembulatan. Yang terakhir menampilkan hasil di textview menggunakan resource format.

Pada baris [45 – 51], pada blok kode ini menghitung fungsi nilai tip dan mengembalikan hasil dalam format mata uang. Nilai tip dengan rumus $(\text{persen}/100) * \text{jumlah}$. Jika pengguna membulatkan maka akan membulatkan ke atas untuk nilai tipnya, lalu hasilnya di format ke bentuk uang dan di return sebagai string.

activity_main.xml:

Pada baris [1 – 8], pada XML ini adalah root untuk seluruh tampilan dengan menggunakan `constraintlayout`, `padding 24dp` semua sisi, dan untuk previewnya di android studio dipakai di `MainActivity`.

Pada baris [10 – 21], ini akan menampilkan layout teks Calculate Tip diatas input dan diposisikan diatas layar dan jarak vertikalnya 120dp

Pada baris [23 – 40], ini akan menampilkan layout input bill dengan icon money dikiri dan padding agar tidak terlalu rapat, user hanya diperbolehkan angka dan decimal.

Pada baris [42 – 53], membuat text input layout dengan border dan akan digunakan membungkus autoCompleteTextView agar seperti input biasa, dan mengubah background dengan warna pink muda

Pada baris [55 – 65], autoCompleteTextView menampilkan dropdown berisi pilihan persen dan icon persen di kiri, panah dropdown di kanan. Ada juga input type none yang artinya tidak bisa costum input atau cukup memilih saja.

Pada baris [69 – 79], menampilkan tombol switch untuk membulatkan keatas atau tidak, diposisikan dibawah dropdown tip dan ada label disebelah kiri dengan “Round Up Tip?”

Pada baris [81 – 92], menampilkan hasil perhitungan tip, dengan ukuran teks besar dan dirampikan dibawah switch rata kiri.

Jetpack Compose:

MainActivity.kt:

Pada baris [1], package com.example.tipcalculatorcompose menyatakan bahwa file berada dalam package ini

Pada baris [3 – 47], adalah seluruh import yang ada di dalam program compose ini

Pada baris [49 – 63], ini adalah class utama aplikasi yang merupakan turunan dari AppCompatActivity, mengatur konten bisa muncul hingga batas layar, menerapkan tema dari TipTimeTheme dan surface yang merupakan komponen material design yang bisa di beri background, padding, dan fillMaxSize() untuk memenuhi layar. Selain itu juga memanggil UI utama yang kita buat di TipTimeLayout()

Pada baris [67 – 77], pada blok kode ini adalah fungsi composable utama yang memiliki beberapa variable seperti input teks tagihan dan menyimpan state selama recompositionn dan mutableStateOf dengan nilai default kosong. State untuk menyimpan apakah tip dibulatkan atau tidak. Membuat daftar pilihan persen tip yang akan ditampilkan di dropdown. Membuat state untuk menyimpan pilihan yang sedang dipilih dropdown dengan default nilai adalah 15%. State yang mengatur apakah dropdown sedang dibuka atau tidak. State yang mengkonversi input ke string double. Mengambil angka dari pilihan dropdown, dan memanggil fungsi calculateTip untuk menghitung nilai tip.

Pada baris [79 – 87], membuat layout vertikal dari atas kebawah, dengan memberi padding otomatis untuk menghindari status bar, diberikan margin kiri kanan 40dp dan isi kolom bisa di scroll kalau melebihi layar. Menyusun kolom semua widget agar ditengah horizontal dan diratakan di tengah secara vertikal.

Pada baris [88 – 93], menampilkan teks dari resource string Calculate Tip yang diberi padding atas dan bawah, serta rata kiri meski kolom center align

Pada baris [94 – 104], memanggil fungsi EditNumberField untuk input jumlah tagihan dengan label bill amount, dan pakai icon gambar money. Selain itu membuat keyboard dikostum jadi tipe angka dan enter menjadi next. Menyambungkan input dan state saat user mengetik, maka amountInput akan diperbarui. Komponen juga diberi padding bawah dan mengisi penuh lebar layar.

Pada baris [105 – 140], ini adalah blok kode yang cukup panjang yang berisikan dropdown menu dari pilihan persentasenya. Ada container untuk dropdown menu untuk mengecek apakah dropdownnya terbuka dan saat di klik dengan status true atau false. Menambahkan margin bawah dan agar box selebar parentnya. Pada TextField menampilkan pilihan 15% karena readOnly, user tidak mengetik tapi memilih dari dropdown. Label pada text untuk membuat nama didalamnya. Trailing icon yang menampilkan ikon panah disamping kanan apakah menu sedang dibuka. menuAnchor() adalah titik untuk posisi dropdown dengan fillMaxWidth untuk selebar mungkin. Dan menggunakan warna default untuk TextField. Daftar dalam opsi dropdown nya ditutup saat klik di luar, dengan membuat loop untuk item dari list options dan juga saat item dipilih ke nilai yang baru maka akan diperbarui nilainya dan dropdown ditutup.

Pada baris [143 – 154], blok kode ini adalah untuk fungsi switch. Memanggil fungsi RoundTheTipRow yang berisi roundUp dengan nilai boolean apakah switchnya menyala. onRoundUpChanged akan dipanggil saat user menyalakan atau mematikan switch. Teks hasil tip akan ditampilkan dengan nilai dari tip dari calculateTip()

Pada baris [157 – 164], pada blok fungsi EditNumberField ini adalah fungsi UI. Membuat label resource ID dari string untuk label dengan menggunakan @StringRes dan leadingIcon untuk resource gambar menggunakan @DrawableRes. Parameter selanjutnya yaitu tipe keyboard, nilai dari input, dan fungsi yang dijalankan saat user mengupdate state.

Pada baris [165 – 174], blok kode ini membuat satu baris input dengan parameter value, dan mengambil ikon disebelah kiri input dari resource dengan painterResource. Membuat style dan fungsi update value. Menampilkan label dari resource string. Mengatur jenis keyboard sesuai dengan parameter.

Pada baris [177 – 181], blok kode ini menyatakan apakah switch dalam kondisi true atau false dengan fungsi callback ketika switch dinyalakan atau dimatikan.

Pada baris [182 – 195], membuat baris selebar dengan parent dan teks sejajar tengah secara vertikal. Pada label ini menampilkan label dari resource dan switch yang diujung kanan dengan kondisi sekarang boolean dan onCheckedChange dijalankan saat user ubah switch.

Pada baris [197 – 203], blok ini membuat fungsi yang berisi amount, tipPercent, dan roundUp. Tip akan dihitung dengan rumus tipPercent dibagi 100 lalu dikali dengan amount. Jika user ingin membulatkan maka roundUp akan true dan dibulatkan ke atas ke bilangan bulat terdekat. Selain itu mengubah format tip jadi string dengan format uang lokal.

Pada baris [205 – 211], memanggil tema dari TipTimeTheme dan memanggil TipTimeLayout untuk ditampilkan sebagai preview.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Akmlsybn/Praktikum-Mobile>

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Kedua hal ini cukup berbeda dalam hal implementasi, contohnya yang paling utama ialah paradigma yang digunakan. Paradigma dari XML sendiri adalah imperative dan Jetpack Compose paradigmanya yaitu declarative. Pada paradigma XML sendiri masih menggabungkan konsep Java + Kotlin, sedangkan pada paradigma Jetpack Compose sudah seluruhnya menggunakan Kotlin.

Kelebihan XML :

1. Pilihan yang bagus untuk memulai sebuah aplikasi android karna mudah dipahami
2. Pemisahan UI dan Logika lebih jelas
3. Tata letak di XML juga bisa digunakan kembali atau reusable

Kekurangan XML :

1. Tata letak yang terlalu banyak, menghasilkan kurangnya efisiensi kode
2. Perlu menulis kode yang panjang
3. Fleksibilitasnya terbatas untuk XML

Kelebihan Jetpack Compose :

1. Sudah menggunakan paradigma deklaratif, yang sangat memudahkan siapapun yang memakainya
2. Mengurangi penulisan kode yang panjang
3. Dapat melihat perubahan UI secara langsung dengan Jetpack Compose.

Kekurangan Jetpack Compose :

1. Perlu pembelajaran lebih giat untuk memahami Jetpack Compose
2. Dokumentasi terbatas karena tergolong cara baru dalam penerapan android
3. Tidak semua bisa menggunakan Jetpack Compose karena masalah kompatibilitas.